PROJECT DOCUMENTATION

# MUFIED Project Implemented Features

This document summarizes the implemented features in the **MUFIED** platform. The summary is based on the current public UI, admin UI, NestJS API, Firebase integration, and related workflows already present in the project.

**3**
Main applications

**7**
Public routes

**2**
Admin workspaces

**4**
Core backend modules

## Public Website Features

The Angular public site delivers the applicant-facing portal and content-driven website.

### Dynamic Landing Page

- Hero section with editable banner, CTA buttons, badge text, and floating information cards.
- Intro, article, programmes, admission path, publication, footer, and contact sections loaded from API content.
- Dedicated events section with event cards, stacked descriptions, optional images, and PDF preview modal.
- Content-driven quick links and mixed internal, external, asset, and PDF link

### Admission Application Flow

- Two-step admission workflow starting with applicant email and date of birth lookup.
- Resume existing applications when an admission already exists for the same email and date of birth.
- Large structured admission form covering student, parents, guardian, address, school, madrassa, and exam details.

handling.

- Photo upload with size validation and live preview.
- State and district support with selectable Indian state and district data.
- Draft persistence in local storage so partially completed applications are not lost.

## Submission Success Experience

- Post-submission page showing application number, hall ticket number, candidate details, venue, and delivery status.
- WhatsApp share link generated automatically after successful submission.
- Direct transition to hall-ticket lookup using the newly submitted application data.

## Hall Ticket System

- Lookup by application number and date of birth.
- Ticket preview screen with candidate photo, personal details, exam center, identification marks, and institutional footer.
- Client-side hall ticket PDF generation using `jsPDF`.
- Separate print and download actions for the generated PDF.

## Exam Result System

- Lookup by application number and date of birth.
- Formatted result sheet preview with watermark, signatures, seal, grade, marks, and result remark.
- Client-side result PDF generation with branded printable layout.
- Separate print and download flows for result documents.

## Documents and Institutions

- Documents page with separate Question Bank and Other Docs sections.
- PDF cards rendered from admin-managed content.
- Institutions page listing Mannaniyya educational institutions and Thahfeelul Qur'an colleges.

# Admin Panel Features

The Angular admin UI provides protected content management and admission operations.

## Admin Authentication

- Dedicated login page for admin access.
- Token-based session handling stored in local storage.
- Route guard enforcement through authenticated admin checks.
- Default admin account bootstrap on first use.

## Website Content Editor

- Section-based editor for hero, intro, programmes, events, documents, hall-ticket, admission, footer, and related content blocks.
- Structured editing for floating cards, stats, CTAs, event descriptions, and footer links.
- Inline JSON editing support alongside section editors.
- Unsaved-change detection and content status messaging.

## Document Management

- Create and manage Question Bank and Other Docs publication cards.
- Upload PDF files to Firebase Storage from admin.
- Direct signed upload flow for large PDF files.
- Delete published documents and remove underlying storage files.

## Event Management

- Create event cards with title, descriptions, optional image, and multiple PDFs.
- Event image uploads with validation.
- Event PDF uploads including direct signed upload support for larger files.
- Delete event images, event PDFs, or full event cards from admin.

## Admission Operations

- List all submitted admissions inside admin.
- Edit admission details, remarks, admission category, and exam-result data.

## Result Management

- Update result subject and mark for each candidate.
- Generate downloadable result marklist PDF from the admissions dataset.

- Delete admissions from the system.
- Export admissions to Excel-compatible spreadsheet format.

- Toggle exam-result menu visibility through managed hall-ticket content settings.

### User Management

- List admin users with role, created date, and creator information.
- Create additional users with roles `admin`, `mentor`, and `student`.
- Restricted login so only users with `admin` role can access the admin UI.

## Backend API Features

The NestJS API powers admissions, public content, admin authentication, storage, and email workflows.

### Admissions API

Create admissions, list them for admin, update and delete records, lookup existing applications, fetch hall tickets, and fetch exam results.

### Application Numbering

Sequential application number generation in the format `MUFIED-YYYY-####`, plus linked hall ticket numbering.

### Duplicate Admission Protection

Prevents duplicate applications for the same email and returns the configured admission authority contact when needed.

### Home Content API

Public content endpoint plus authenticated admin endpoint for updating full home-page content.

### Admin Auth API

Admin login, session validation, current-admin lookup, and protected admin routes using bearer token validation.

### Admin Users API

Protected listing and creation of admin-side users with normalized usernames and hashed passwords.

## Storage, Delivery, and Platform Features

The project includes operational capabilities beyond basic CRUD.

### Firebase Realtime Database

Admissions, home content, admin users, admin sessions, and sequence counters are persisted in Firebase.

### Firebase Storage Uploads

Public PDFs and event images are uploaded to storage, published for browser access, and cleaned up on delete.

### Signed Upload URLs

Large PDFs can be uploaded directly from the browser to storage through signed write URLs.

### Email Confirmation

Admission submission can send confirmation emails through SMTP when configured, while still succeeding if email is unavailable.

### Static Content Fallback

If Firebase is unavailable, the project can still serve seeded static home content through the API.

### Content Migration and Normalization

Stored home content is normalized and migrated to newer structure automatically when older data shapes are detected.

## Implemented Roles and Managed Areas

admin mentor student

Landing Content Admissions Hall Tickets Exam Results Documents Events Users

## Scope Note

This document only includes features that are visibly implemented in the current `mufied` repository. It does not claim extra modules or integrations unless they are present in the source code.

Public feature summary for the MUFIED platform.